

CAREERBRIDGE – DAY 9 EXERCISE

Application Workflow and Applicant Status Management

Project: CareerBridge – Job & Internship Portal

Week: 2

Day: 9

Student Name: Larbi Peniels Okyerewa

Date: 9/17/2026

PART B — REVIEW THE APPLICATION TABLE

Review the Application table created during Week 1.

It may contain fields similar to:

ApplicationID

JobID

JobSeekerID

CoverLetter

CVPath

ApplicationStatus

AppliedOn

UpdatedOn

Use your actual database design.

Identify:

Primary Key: ApplicationID

Foreign Key to Job: JobID

Foreign Key to Job Seeker: JobSeekerID

Column containing the status: Status

Column containing application date: ApplicationDate

PART C — UNDERSTAND THE RELATIONSHIP

Recall the ERD from Day 3.

JobSeeker

|
| 1
|
| *

Application

|
| *
|
| 1

Job

A Job Seeker can have many Applications.

A Job can receive many Applications.

The Application entity connects the two.

Question

Why do we need the Application table instead of storing JobID directly inside the Job Seeker table?

If we stored JobID directly in the Job Seeker table, a Job Seeker would only be able to have one JobID at a time, which would not allow them to apply for multiple jobs.

PART E — DESIGN THE APPLICATION API

CareerBridge should support endpoints similar to:

Method	Endpoint	Purpose
POST	/api/jobs/:jobId/apply	Apply for a job

Method	Endpoint	Purpose
GET	/api/applications/me	My applications
GET	/api/applications/:id	View an application
GET	/api/jobs/:jobId/applicants	Employer views applicants
PATCH	/api/applications/:id/status	Update applicant status

Decide which role should access each endpoint.

PART G — DO NOT TRUST JOBSEEKERID FROM THE CLIENT

Do not design the request like this:

```
{
  "jobSeekerId": 25,
  "jobId": 15
}
```

and blindly trust those IDs.

CareerBridge already has authentication.

The server should obtain the logged-in user from the verified JWT.

Conceptually:

Bearer Token



authMiddleware



req.user



UserID



Find Job Seeker Profile



Create Application

Question

Why is accepting an arbitrary jobId from Postman dangerous?

It is dangerous because a user could provide another person's jobId and submit an application on their behalf.

PART H — ROLE VALIDATION

Only a Job Seeker should normally apply for a job.

Test:

JobSeeker → POST /apply

Expected:

Allowed

Test:

Employer → POST /apply

Rejected

PART J — CHECK JOB AVAILABILITY

A job should not automatically accept applications simply because its JobID exists.

Check relevant rules such as:

Status = Active

and:

ApplicationDeadline >= Current Date

If the job is closed:

Application rejected

If the deadline has passed:

Application rejected

Question

Why should this validation happen in the backend rather than only in React?

Because React runs on the client side and can be bypassed or manipulated.

PART K — PREVENT DUPLICATE APPLICATIONS

A Job Seeker should not apply for the same job multiple times.

Before inserting:

JobID + JobSeekerID

↓

Check Application

↓

Already exists?

↙ ↘

Yes No

↓ ↓

Reject Insert

Write the SQL concept you could use to check:

-- Write your query here

SELECT ApplicationID

FROM Application

WHERE JobID = @JobID

AND JobSeekerID = @JobSeekerID;

PART L — DATABASE-LEVEL DUPLICATE PROTECTION

Application code can check for duplicates, but the database can also help.

Think about a UNIQUE constraint involving:

JobID

+

JobSeekerID

Question

Why might it be useful to have both:

1. An application-level duplicate check, and
2. A database UNIQUE constraint?

The application-level check allows the API to give a clear response when a duplicate application is detected. The database UNIQUE constraint provides an additional layer of protection and prevents duplicate records even if application logic is bypassed or two requests arrive almost simultaneously.

PART N — WHY STATUS MUST BE SERVER-CONTROLLED

Consider this request:

```
{  
  "coverLetter": "Please consider my application.",  
  "applicationStatus": "Accepted"  
}
```

Should CareerBridge allow a Job Seeker to submit themselves as Accepted?

No

Explain:

The Job Seeker should not be allowed to choose their own application status.

PART P — DO NOT USE USERID QUERY PARAMETERS

Avoid designs such as:

GET /api/applications?userId=25

for the Job Seeker's own applications.

Prefer:

GET /api/applications/me

Why?

GET /api/applications/me is safer because the server identifies the Job Seeker from the authenticated JWT. This prevents a user from changing a userId in the URL and attempting to access another user's applications.

PART Q — VIEW ONE APPLICATION

Create:

GET /api/applications/:id

Example:

GET /api/applications/51

But simply knowing ApplicationID = 51 must not automatically give access.

CareerBridge should check who is requesting it.

Discuss who should be allowed to view an application:

The Job Seeker who submitted it

Employer who owns the related job

Administrator

Explain your choices:

The Job Seeker who submitted the application should be able to view it. The Employer who owns the related job should also be able to view it because they are responsible for reviewing applicants. An Administrator can view it for administrative purposes. Other Employers and unrelated Job Seekers should not have access because they do not own or manage the application.

PART S — EMPLOYER OWNERSHIP

Before returning applicants:

Authenticated Employer



Requested Job



Who owns Job?



Same Employer?



No



403



Yes



Return

Applicants

Employer B must not be able to change:

`/api/jobs/15/applicants`

to:

`/api/jobs/10/applicants`

and see Employer A's applicants.

Question

Which Day 8 concept are we reusing here?

Ownership authorization concept

PART U — WHY PATCH?

Today we are updating only one part of the Application resource:

ApplicationStatus

Research and explain:

What is PATCH?

Patch is an HTTP method used to partially update an existing resource.

How is PATCH different from PUT?

PUT is normally used to replace or update the complete resource, while PATCH is used to make a partial update to a resource.

PART W — WHO CAN UPDATE STATUS?

Should a Job Seeker be able to execute:

PATCH /api/applications/51/status

No

Should any Employer be able to do it?

No

Who should normally be allowed?

The Employer who owns the job associated with the application.

PART X — STATUS TRANSITION RULES

A stronger implementation should consider not just whether a status exists, but whether the transition makes sense.

For example:

Submitted



Reviewed



Shortlisted

↙ ↘

Accepted Rejected

Discuss whether CareerBridge should allow:

Submitted → Accepted

Your decision:

No

What about:

Rejected → Shortlisted

Your decision:

No

Explain:

Application statuses should follow a logical workflow. A normal flow is:

Submitted → Reviewed → Shortlisted → Accepted/Rejected

PART Y — APPLICATION HISTORY CHALLENGE

Consider what happens if:

Submitted

↓

Reviewed

↓

Shortlisted

↓

Accepted

If the Application table stores only:

ApplicationStatus = Accepted

we know the current status but may not know when each previous change occurred.

Challenge Question

How could you design CareerBridge so that status changes are historically recorded?

Proposed entity/table:

ApplicationStatusHistory

Possible fields:

HistoryID

ApplicationID

OldStatus

NewStatus

ChangedBy

ChangedOn

PART Z — SQL JOINS

For Employer applicant lists, you may need information from:

Application



Job



Company

Application



JobSeeker



User

Draw or describe the JOIN path:

Application



Job



Company

For the applicant:

Application



JobSeeker



User

HTTP STATUS CODE EXERCISE

Complete the table.

Situation	HTTP Status
Application created	201 Created
Applications retrieved	200 OK
Missing required data	400 Bad Request
No JWT	401 Unauthorized
Wrong role	403 Forbidden
Job not found	404 Not Found
Application not found	403 Forbidden

Employer tries to view another employer's applicants _____

Situation	HTTP Status
Duplicate application	409 Conflict
Unexpected server error	500 Internal Server Error

NEGATIVE TESTING

Test the API against incorrect or unauthorized actions.

Test 1

Job Seeker applies twice to the same job.

Expected:

409 Conflict – Duplicate application

Test 2

Job Seeker applies to a nonexistent job.

Expected:

404 Not Found – Job does not exist

Test 3

Job Seeker applies to a closed job.

Expected:

400 Bad Request – Job is unavailable

Test 4

Job Seeker applies after the deadline.

Expected:

400 Bad Request – Application deadline has passed

Test 5 — Employer attempts to apply

Expected:

403 Forbidden — Only Job Seekers can apply.

Test 6 — Job Seeker attempts to change their status to Accepted

Expected:

403 Forbidden — Job Seeker is not authorized to change application status.

Test 7 — Employer views another Employer's applicants

Expected:

403 Forbidden — Employer does not own the job.

Test 8 — Employer updates application belonging to another Employer's job

Expected:

403 Forbidden — Employer does not own the related job.

Test 9 — Employer sends invalid status

Expected:

400 Bad Request — SUPERSTAR is not an allowed application status.

THINKING QUESTIONS

1. Why shouldn't JobSeekerID come directly from Postman when applying?

Because the client could provide another person's JobSeekerID and submit an application on their behalf. The server should derive the Job Seeker identity from the authenticated JWT.

2. How does CareerBridge know which Job Seeker is applying?

Answer:

The authentication middleware verifies the JWT and places the authenticated user's information in req.user. The server uses that information to identify the Job Seeker.

3. Why must duplicate applications be prevented?

Answer:

To prevent the same Job Seeker from submitting multiple applications for the same job and creating incorrect or duplicate records.

4. Why should the backend check the application deadline?

Answer:

Because the backend must enforce the business rule that applications cannot be submitted after the deadline. Client-side checks can be bypassed.

5. What is the difference between authentication and ownership?

Answer:

Authentication determines who the user is.

Ownership determines whether that authenticated user has permission to access or modify a particular resource.

6. Why can't every Employer see every application?

Answer:

Applications contain information belonging to specific jobs and applicants. An Employer should only access applications for jobs that the Employer owns.

7. Why should application status be controlled by the server?

Answer:

To prevent applicants from manipulating their own status, such as changing it from Submitted to Accepted.

8. What is the difference between PUT and PATCH?

Answer:

PUT is generally used to replace or update an entire resource, while PATCH is used to make a partial update.

9. Why might an ApplicationStatusHistory table be useful?

Answer:

It records previous status changes, including the old status, new status, person who made the change and date/time. This provides a history of the application's workflow.

10. Why should important authorization checks happen in the API even if React hides the buttons?

Answer:

Because React runs on the client and can be bypassed. The API must independently enforce authorization so unauthorized users cannot access protected endpoints directly.